

MLE5004: Computer systems architecture

V. Overflow concept analysis.

- **CF (Carry Flag)** is the transport flag. It will be set to 1 if in the Last Performed Operation (LPO) there was a transport digit outside the representation domain of the obtained result and set to 0 otherwise. For example, in the addition

$$\begin{array}{r} 1001\ 0011 \\ + 0111\ 0011 \\ \hline 1\ 0000\ 0110 \end{array}$$

or in base 10, $147 + 115 = 262$, there is transport and CF is set to 1. **CF**

flags the UNSIGNED overflow!

- **OF (Overflow Flag)** flags the signed overflow. If the result of the LPO (considered in the signed interpretation) didn't fit the reserved space (admissible representation interval), then OF will be set to 1 and will be set to 0 otherwise. For the above example, $OF=0$

An overflow is the situation/condition which expresses the fact that the result of an operation didn't fit the reserved space for it. (neither -147 "fits" in the range $[-128..+127]$ nor on a single byte, but it is more difficult to intuit that the definition includes and refers to this case as well..)

Definition 5.1. At the level of the assembly language an overflow is a situation/condition which expresses the fact that the result of the LPO didn't fit the reserved space for it OR does not belong to the admissible representation interval for that size OR that operation is a mathematical nonsense in that particular interpretation (signed or unsigned).

5.1 CF vs. OF, the overflow concept

If we consider the following examples:

- in binary representation $\begin{array}{r} 1001\ 0011 \\ + 1011\ 0011 \\ \hline 1\ 0100\ 0110 \end{array}$

- base 16 representation $\begin{array}{r} 93 \\ + B3 \\ \hline 1\ 46 \end{array}$

- unsigned interpretation $\begin{array}{r} 147 \\ + 179 \\ \hline 326 \end{array}$

- signed interpretation
$$\begin{array}{r} -109 \quad + \\ -77 \\ \hline -186 \end{array}$$

326 and -186 are the correct results in base 10 for the corresponding interpretations for the above operands.

From the discussion on the admissible representation intervals and respectively from the analysis of the type of problems "Which is the minimum number of bits on which we can represent the numbers 326 and -186 ?" it results that:

- $326 \in [0, 511]$ and $-186 \in [-256, +255]$ for eight bits, and thus the minimum number of bits on which 326 and -186 can be represented is 9, and the representation of -186 is $512 - 186 = 326 = (146)_{16} = (101000110)_2$.

As a result, all the above operations are mathematically correct on 9 bits and the final results cannot fit in the reserved space. The above addition is performed at byte level (since in the assembly language we have $\text{ADD byte} + \text{byte} \rightarrow \text{byte}$) and as a result, the result is wrong.

This is signaled simultaneously by the flags CF (for unsigned interpretation) and OF (for signed interpretation), $CF = OF = 1$.

As a result:

- in binary representation
$$\begin{array}{r} 1001\ 0011 \quad + \\ 1011\ 0011 \\ \hline 1\ 0100\ 0110 \end{array}$$

- base 16 representation
$$\begin{array}{r} 93 \quad + \\ B3 \\ \hline 1\ 46 \end{array}$$

- unsigned interpretation
$$\begin{array}{r} 147 \quad + \\ 179 \\ \hline 70 \quad \text{and } CF=1 \end{array}$$

- signed interpretation
$$\begin{array}{r} -109 \quad + \\ -77 \\ \hline +70 \quad \text{and } OF=1 \end{array}$$

By setting both CF and OF to 1, the "message" from the processor is that in both interpretations in base 10 of the above base 2 addition the result is incorrect.

Consider the following numerical example

- in binary representation
$$\begin{array}{r} 0101\ 0011 \quad + \\ 0111\ 0011 \\ \hline 1100\ 0110 \end{array}$$

- base 16 representation
$$\begin{array}{r} 53 \quad + \\ 73 \\ \hline C6 \end{array}$$

- unsigned interpretation
$$\begin{array}{r} 83 \quad + \\ 115 \\ \hline 198 \end{array}$$

- signed interpretation
$$\begin{array}{r} 83 \quad + \\ 115 \\ \hline \text{!!!}198 \end{array}$$

198 is the correct result in base 10 for unsigned interpretation, now we have to see if the result fits on 8 bits, yes – it does therefore we will have $CF = 0$. And if the result of the binary operation in signed interpretation is correct when interpreted in base 10, it is not, because $(11000110)_2$ is not a positive number in the signed interpretation!). So, what we will get, as results represented on 1 byte, will be:

- in binary representation
$$\begin{array}{r} 0101\ 0011 \quad + \\ 0111\ 0011 \\ \hline 1100\ 0110 \end{array}$$

- base 16 representation
$$\begin{array}{r} 53 \quad + \\ 73 \\ \hline C6 \end{array}$$

- unsigned interpretation
$$\begin{array}{r} 83 \quad + \\ 115 \\ \hline 198 \quad \text{and } CF = 0 \end{array}$$

- signed interpretation
$$\begin{array}{r} 83 \quad + \\ 115 \\ \hline -58 \quad \text{and } OF = 1 \end{array}$$

Setting $CF = 0$ expresses the fact that the unsigned interpretation in base 10 of the above base 2 addition is correct and the operation functions properly both at the level of the interpretation of the operands and the result. OF will however be set by the processor to the value 1, this meaning that the signed interpretation of the base 2 addition above reflects an incorrect operation, more precisely the result obtained is an incorrect one in signed interpretation.

OF will be set to 1 (signed overflow) if for the addition operation we are in one of the following two situations (overflow rules for addition in signed interpretation). These are the only 2 situations that can issue overflow status for the **addition** operation:

$$\begin{array}{r} 0 \dots\dots + \\ 0 \dots\dots \\ \hline 1 \dots\dots \end{array} \quad \text{or} \quad \begin{array}{r} 1 \dots\dots + \\ 1 \dots\dots \\ \hline 0 \dots\dots \end{array}$$

Semantically, in the above two situations denote the impossibility of mathematical acceptance of the 2 operations: we cannot add two positive numbers and obtain a negative result and we cannot add two negative numbers and obtain a positive result

In the case of **subtraction**, we also have two overflow rules in the signed interpretation, a consequence of the two overflow rules for addition:

$$\begin{array}{r} 1 \dots\dots - \\ 0 \dots\dots \\ \hline 0 \dots\dots \end{array} \quad \text{or} \quad \begin{array}{r} 0 \dots\dots - \\ 1 \dots\dots \\ \hline 1 \dots\dots \end{array}$$

Semantically, the two above situations denote the impossibility of mathematical acceptance of the two operations: we cannot subtract a positive number from a negative one and obtaining a positive one, neither can we subtract a negative number from a positive one and obtaining a negative one).

Consider the following examples, where -102 and 154 are the correct mathematical results:

- in binary representation
$$\begin{array}{r} \text{1 } 0110\ 0010 \quad - \\ 1100\ 1000 \\ \hline 1001\ 1010 \end{array}$$

- base 16 representation
$$\begin{array}{r} 62 \quad - \\ \text{C8} \\ \hline 9\text{A} \end{array}$$
- unsigned interpretation
$$\begin{array}{r} 98 \quad - \\ 200 \\ \hline -102 \end{array}$$
- signed interpretation
$$\begin{array}{r} 98 \quad - \\ -56 \\ \hline 154 \end{array}$$

However, if we also take into account the **interpretations of the RESULT** obtained in the program, we have:

- in binary representation
$$\begin{array}{r} 1 \ 0110 \ 0010 \quad - \\ 1100 \ 1000 \\ \hline 1001 \ 1010 \end{array}$$
- base 16 representation
$$\begin{array}{r} 62 \quad - \\ \text{C8} \\ \hline 9\text{A} \end{array}$$
- unsigned interpretation, we need significant digit borrowing for performing the subtraction, therefore $CF = 1$
$$\begin{array}{r} 98 \quad - \\ 200 \\ \hline 154 \end{array}$$
 and **CF = 1**
- signed interpretation
$$\begin{array}{r} 98 \quad - \\ -56 \\ \hline -102 \end{array}$$
 and **OF = 1**

Both interpretations of the result obtained in base 2 are mathematically **incorrect**, so CF and OF will both be set to the value 1.

None of the interpretations above are mathematically correct, because in base 10, the subtraction $98 - 200$ should equal -102 , but instead 154 is given as result in unsigned interpretation - which is an incorrect result! In signed interpretation we have $98 - (-56) = -102$, again an incorrect result, instead of the correct one $98 - (-56) = 154$.

In conclusion, in order to be mathematically correct, the results of the two above subtractions should be switched between the two interpretations. The two interpretations associated to the binary subtraction are both mathematically incorrect. For this reason the 80x86 microprocessor will set $CF = 1$ and $OF = 1$.

The multiplication operation does not produce overflow at the level of 80x86 architecture, the reserved space for the result being enough for both interpretations. Anyway, even in the case of multiplication, the decision was taken to set both $CF = OF = 0$, in the case that the size of the result is the same as the size of the operators (*byte * byte = byte*, *word * word = word* or *(double word) * (double word) = double word*). In the case that *byte * byte = word*, *word * word = double word* or *(double word) * (double word) = quad word*, then $CF = OF = 1$.

The worst effect of overflow is in the case of the division operation, here we are talking about integer division: if the quotient does not fit in the reserved space then the « division overflow » will signal a 'Run-time error' and the operating system will stop the running program and will issue one of the 3 semantic equivalent messages: 'Divide overflow', 'Division by zero' or 'Zero divide'. The space reserved by the assembler for the result in case of a division operation is:

- a *byte* for the division $\frac{\text{word}}{\text{byte}}$,
- a *word* for the division $\frac{\text{double word}}{\text{word}}$,
- a *double word* for division $\frac{\text{quad word}}{\text{double word}}$.

In case of a correct division, *CF* and *OF* are undefined. If we have a division overflow, the program crashes, the execution stops and of course it doesn't matter which are the values from *CF* and *OF*.

Consider the following example:

$$\frac{\text{word}}{\text{byte}} \rightarrow \text{byte}, \quad \frac{1002}{3} = 334 = \text{division overflow} - \text{fatal} - \text{Run time error.}$$

The error message is: '*Divide overflow*', '*Division by zero*' or '*Zero divide*' - technically an INT 0 will be issued. *Why do we get such a message if we divide by 3?* Because regardless of whether we divide by zero or something different from zero, the result doesn't fit in the reserved space. The end result is the same: run-time error because the result of the integer division doesn't fit the allocated space, 334 does not fit in a byte.

$\frac{\text{number}}{0}$ - Zero divide = it is an overflow because ∞ does not fit in the reserved space...

Why do we need CF and OF in EFLAGS simultaneously? Isn't a single flag enough to show us if we have an overflow or not, either in the signed or in unsigned interpretation? No, because when performing an addition or subtraction operation in base 2, in fact two operations are performed simultaneously in base 10: one in the signed interpretation and the other in the unsigned interpretation. As a result, two different flags are needed simultaneously to deal with each interpretation in base 10 separately: *CF* - for the unsigned interpretation and *OF* - for the signed one.

This is the case because the operation of addition or subtraction done in base 2 is performed identically, therefore, regardless of the signed or unsigned interpretation of the operands and the result! This is also the reason why there are no *IADD* or *ISUB* instructions in assembly language.

So, why do we need then IMUL and IDIV? Because unlike addition and subtraction, which work the same way in base 2 regardless of the interpretation (signed or unsigned), signed and unsigned multiplication and division work differently.

As a result, in case of addition and subtraction there is no need to specify before the computation how we want the operands and the result to be interpreted. The two operations are performed the same way in binary regardless of the interpretation. It is enough to decide after performing the operation how we want the operands and the result to be interpreted.

In contrast, for multiplication and division there is the need to specify before the operation how we want the operands to be interpreted. This is done by specifying the instruction:

- *MUL* and *DIV* if we want to interpret the operands as unsigned numbers,
- *IMUL* and *IDIV* if we want to interpret the operands as signed numbers.

References

1. A Vancea. Courses, (tank you :))
2. Cavanagh, J. (2013). X86 Assembly Language and C Fundamentals (1st ed.). CRC Press. <https://doi.org/10.1201/b14582>
3. Randall Hyde. (2020). The Art of Assembly Language (2nd Edition). No Starch Press.
4. Daniel Kusswurm. (2023). Modern X86 Assembly Language Programming: 32-bit, 64-bit, SSE, and AVX. Apress.